1 Computer System Overview

Main Elements and Registers

Processor	executes instructions and controls system operation.
Main memory	stores currently used programs and data.
I/O modules	move data between the computer and external devices.
System bus	carries data, address, and control signals among modules.

User-visible registers: referenced by machine instructions to reduce memory access. **Control/status registers**: PC, IR, MAR, MBR, PSW; used by CPU and OS to control execution.

Instruction Actions

- **Processor-memory**: load/store between CPU and memory.
- **Processor-I/O**: transfer data between CPU and I/O modules.
- **Data processing**: arithmetic or logical operations.
- **Control**: branch, jump, call, return; changes execution order.

Interrupts

Purpose and Types

Interrupt: a mechanism by which I/O, timer, program error, or hardware failure interrupts normal processor execution. Main purpose: improve processor utilization.

Program	divide by zero, overflow, illegal instruction.
Timer	lets the OS regain control periodically.
I/O	device reports completion or readiness.
Hardware failure	power failure, memory parity error.

Processing Sequence

1. Device issues interrupt; CPU finishes current instruction.
2. CPU acknowledges, saves PC and PSW, and loads ISR address.
3. ISR saves remaining state, handles the event, restores state.
4. Old PC and PSW are restored; interrupted program resumes.

PC tells where to resume; PSW records processor state. Polling repeatedly tests device flags; interrupts let the device notify the CPU.

Multiple Interrupts

Sequential: disable interrupts during ISR; new interrupts stay pending. Simple but may delay urgent events.

Nested: assign priorities; a higher-priority interrupt may interrupt a lower-priority ISR.

Memory, Cache, I/O

Memory Hierarchy

Higher levels are **faster**, **smaller**, and **more expensive per bit**; lower levels are larger and slower. Distinguish levels by capacity, access time, cost per bit, and frequency of access.

$$T_{\text{avg}} = \sum_i \left(\prod_{j<i} (1 - H_j) \right) T_i$$

$$C_{\text{avg}} = \sum C_i S_i / \sum S_i$$

Cache and Locality

Cache: small fast memory between processor and main memory; stores recently or frequently used blocks to reduce average access time.

Temporal locality: a referenced item is likely to be referenced again soon; keep recent items in fast memory.

Spatial locality: nearby addresses are likely to be referenced soon; fetch a block, not only one word.

I/O and DMA

Programmed I/O: CPU waits by polling flags such as FGI/FGO.

Interrupt-driven I/O: IEN enables the device to interrupt when ready.

DMA: transfers data directly between I/O device and memory; DMA gets high memory priority because device delay may cause data loss, while CPU delay usually only slows execution.

Chapter 1 Exam Questions

Review 1.1–1.10

Q: List the four main computer elements.
A: Processor, main memory, I/O modules, system bus.
Q: Define the two register categories.
A: User-visible registers; control and status registers.
Q: What is an interrupt?
A: A mechanism that lets another module interrupt normal CPU execution so an ISR can handle the event.
Q: How are multiple interrupts handled?
A: Sequential processing disables interrupts during ISR; nested processing uses priorities.
Q: What distinguishes memory hierarchy levels?
A: Capacity, access time, cost per bit, and access frequency.
Q: Multiprocessor vs multicore?
A: Multiprocessor has multiple processors; multicore has multiple cores on one chip.

Problem Stems and Answers

1.1 I/O program: Load dev5, add M[940], store dev6: dev6 receives `0005`.
1.2 MAR/MBR: Expand Fig. 1.4 execution: fetch via MAR/MBR; final M[941] = `0005`.
1.3 32-bit instr.: 24-bit address field gives 16 MB; PC = 24 bits, IR = 32 bits.
1.4 16-bit addr.: 16-bit memory: 128 KiB; 8-bit memory: 64 KiB; 8-bit port number: 256 ports.
1.5 bus rate: 8 MHz, 16-bit bus, 4 clocks/bus cycle gives 4 MB/s; wider bus or doubled clock gives 8 MB/s.
1.6 Teletype I/O: FGI/FGO polling wastes CPU; IEN enables interrupt-driven I/O.
1.7 DMA priority: DMA gets memory priority because device delay may lose data; CPU delay mostly slows execution.
1.8 DMA slowdown: 9600 bps DMA slows 1 MIPS CPU by about 0.12%.
1.9 I/O rate: Programmed I/O = 25,000 words/s; DMA = 2.15×10^6 words/s.
1.10 locality code: Spatial: sequential `a[i]`; temporal: repeated same `a[i]` in inner loop.
1.11 n-level memory: Use $T_{\text{avg}} = \sum_i (\prod_{j<i}(1 - H_j))T_i$, $C_{\text{avg}} = \sum C_i S_i / \sum S_i$.
1.12 cost/hit: Main memory ≈ \$83.89; cache-tech ≈ \$838.86; $H \approx 99.17\%$.
1.13 cache/disk: Average access time ≈ 480,026 ns.
1.14 stack PC: No; the PC cannot generally be eliminated by using the stack top.

2.1 OS Objectives and Functions

Definition

Operating system: a program that controls application execution and acts as an interface between applications and computer hardware.

Three Objectives

Convenience	makes the computer easier to use.
Efficiency	uses CPU, memory, I/O, files, storage, and network resources efficiently.
Evolution	permits new functions, tests, fixes, and hardware support without disrupting service.

Two Main Roles

- **User/computer interface**: hides hardware details and provides usable services.
- **Resource manager**: allocates processor, memory, I/O devices, files, storage, and network resources.

OS Services

Program development	editors, debuggers, compilers, libraries.
Program execution	loads code/data, initializes files, I/O, and resources.
I/O access	uniform device interface, e.g. `read()`, `write()`.
File access	file organization, permissions, protection.
System access	login, authorization, resource contention control.
Error handling	detect hardware/software errors; terminate, retry, or report.
Accounting	collect usage/performance data; support billing and tuning.

Interfaces: ISA vs ABI vs API

ISA	CPU instruction set	hardware/software boundary; user ISA vs system ISA.
ABI	binary compatibility	system-call interface, registers, calling convention, binary format.
API	source-level calls	library functions used by programmers, e.g. `open()`, `malloc()`.

Memory hook: API = coding; ABI = compiled binary execution; ISA = CPU instructions.

Kernel, Control, Evolution

Kernel / Nucleus

Kernel: central OS part, normally resident in main memory, containing the most frequently used OS functions: process, memory, I/O, file, interrupt, and system-call handling.

How the OS Controls the CPU

The OS is also software. It gains control, schedules and allocates resources, gives control to user programs, then regains control through interrupts, system calls, or exceptions.

Why OSs Must Evolve

- **Hardware upgrades**: new processors/devices require new support.
- **New services**: users and administrators need new tools and features.
- **Fixes**: bugs must be repaired without destabilizing old services.

System Calls and Modes

System Calls

System call: controlled entry point for a user program to request OS services such as file I/O, process creation, memory allocation, communication, and protection.

Dual-Mode Operation

User mode	low privilege; applications cannot directly execute privileged instructions.
Kernel mode	high privilege; OS kernel performs protected operations.

Flow: a user program requests a system call/trap; the kernel performs the service and returns to user mode.

PDF Stems and Answers

R2.1: Stem: three OS-design objectives. A: convenience, efficiency, evolution.
R2.2: Stem: kernel of an OS. A: memory-resident core with frequent OS functions.
P2.4: Stem: system-call purpose and dual-mode relation. A: safe service request; user mode traps to kernel mode, then returns.
P2.5: Stem: OS/390 SRM checks untouched page frames. A: find inactive pages, guide replacement/reallocation, detect memory pressure or thrashing.
R2.6: Stem: five storage duties. A: isolation, allocation, modular support, protection/access control, long-term storage.

Chapter 2 Fast Recall

- OS = application controller + hardware interface + resource manager.
- OS hides hardware details behind services and interfaces.
- Important services: development, execution, I/O, files, access control, errors, accounting.
- Kernel holds common privileged OS functions in memory.
- System calls are the legal bridge from user mode to kernel mode.
- Evolution needs modular design, clear interfaces, and maintainability.

3 Process Description and Control

Process, Image, PCB

Process: a program in execution; an entity that can be assigned to and executed by a processor.
Process image: program code, data, stack, and process attributes.
PCB: OS record holding enough information to stop and later resume a process as if it had not been interrupted.

ID	PID, parent PID, user ID.
CPU state	registers, PC, PSW, stack pointer.
Control	state, priority, scheduling links, memory pointers, I/O status, accounting.

Instruction trace: sequence of instructions actually executed by a process.
OS tables: memory, I/O, file, and process tables.

Five-State Model

New	created but not yet admitted.
Ready	can run once assigned a CPU; lacks only CPU.
Running	currently executing on the CPU.
Blocked	waits for an event such as I/O completion.
Exit	terminated or released.

Admit: New → Ready. **Dispatch**: Ready → Running. **Timeout/preempt**: Running → Ready. **Event wait**: Running → Blocked. **Event occurs**: Blocked → Ready. **Release**: Running → Exit.

Ready = waiting for CPU; Blocked = waiting for an event. An event-completed blocked process returns to Ready, not directly to Running.

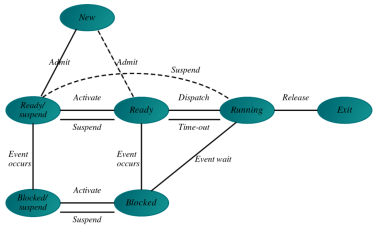
Suspension and Swapping

Swapping: moving a process between main memory and disk to free memory and improve CPU use.
Suspended: not immediately available for execution; may or may not be waiting for an event.

Ready/Suspend	swapped out; event condition satisfied; can run after swap-in.
Blocked/Suspend	swapped out and still waiting for an event.
Reasons	swapping, OS reason, interactive user request, timing, parent request.
Traits	not executable now; may wait on event; agent-suspended; needs explicit activation.

Queues: use one Ready queue plus per-event Blocked queues; suspended states use Ready/Suspend and per-event Blocked/-Suspend queues.

Seven-State Diagram



Control, Modes, Switching

Creation steps: assign PID; allocate process space; initialize PCB; link into ready or suspend queues; create/expand accounting and other tables.
User mode: low privilege; cannot execute privileged instructions. **Kernel/System mode**: high privilege for protected OS operations.
Interrupt: asynchronous external event. **Trap**: exception caused by the current instruction. **Supervisor call**: explicit request for an OS service.
Mode switch: changes privilege mode, possibly same process. **Process switch**: CPU changes from one process to another.
Process switch work: save old PC, PSW, registers into PCB; update state and queues; choose next process; load next PCB/context.

Chapter 3 Review Q&A

R3.1 Trace: Stem: instruction trace. A: executed instruction sequence of a process.
R3.2 Creation: Stem: process creation events. A: new batch job, interactive logon, OS-created service, spawned child.
R3.3 States: Stem: Fig. 3.6 states. A: New, Ready, Running, Blocked, Exit.
R3.4 Preempt: Stem: preempt a process. A: reclaim CPU before the process voluntarily yields or finishes.
R3.5 Swapping: Stem: swapping and purpose. A: move process memory to/from disk to free memory and improve CPU use.
R3.6 Two blocked: Stem: why Fig. 3.9b has two blocked states. A: resident Blocked vs swapped-out Blocked/Suspend.
R3.7 Suspended: Stem: four suspended-process traits. A: not executable now; may wait; agent-suspended; explicitly activated.
R3.8 OS tables: Stem: OS management tables. A: memory, I/O, file, process.
R3.9 PCB info: Stem: three PCB categories. A: identification, processor state, process control.
R3.10 Modes: Stem: why user/kernel modes. A: protect OS, PCBs, memory, I/O, privileged instructions.
R3.11 Create: Stem: OS steps to create process. A: PID, space, PCB, links, other tables.
R3.12 Int/trap: Stem: interrupt vs trap. A: external asynchronous event vs instruction-caused exception.
R3.13 Examples: Stem: three interrupts. A: clock, I/O, memory fault.
R3.14 Switches: Stem: mode switch vs process switch. A: privilege change vs changing the running process.

Chapter 3 Problem Q&A

P3.1 Transitions: Stem: R/RUN/B/NR transition table. A: 1 dispatch; 2 preempt; 3 event wait; 4 event occurs; 5 ready swap-out; 6 blocked swap-out.
P3.2 Timeline: Stem: process states at $t = 22, 37, 47$. A: $t = 22$: P1 B(d3 read), P3 B(d2 read), P7 B(d3 write), P5 likely running. $t = 37$: P1/P3 ready, P5 Blocked/Suspend, P7 blocked, P8 running. $t = 47$: P5 ready, P7 blocked, P8 exit; one ready process may run.
P3.3 Seven states: Stem: possible/impossible Fig. 3.9b transitions. A: Possible direct moves: N→R/RS/E; R→Run/RS/E; Run→R/B/RS/E; B→R/BS/E; RS→R/E; BS→B/RS/E; all other direct moves are impossible.
P3.4 Queues: Stem: seven-state queueing diagram. A: Ready queue, processor, per-event Blocked queues, Ready/Suspend queue, and per-event Blocked/Suspend queues.
P3.5 Policy: Stem: choose Ready vs high-priority Ready/Suspend. A: swap in Ready/-Suspend only when priority or aging benefit exceeds swap cost; otherwise dispatch resident Ready.
P3.6 VAX states: Stem: justify VAX/VMS wait states. A: many waits give precise event/resource queues; paging/global resource waits need no resident/outswapped pair; transitions follow dispatch, wait, event, swap.
P3.7 Access modes: Stem: four access modes vs two. A: Kernel, Executive, Supervisor, User give finer least-privilege protection, but add complexity and mode-switch overhead.
P3.8 Rings: Stem: ring protection need-to-know problem. A: rings are nested; inner domains can access outer objects, so selective need-to-know access is hard to enforce.
P3.9 Multi-wait: Stem: process waiting on multiple events. A: yes, e.g., network, keyboard, timeout; use wait descriptors or multiple queue links and remove stale links after wakeup.
P3.10 Fixed saves: Stem: fixed interrupt register-save locations. A: practical only for simple nonnested systems; nested interrupts, multiprogramming, and switches need PCB/kernel-stack context.
P3.11 Real-time: Stem: nonpreemptive kernel and real-time. A: it can cause unbounded latency, priority inversion, and missed deadlines; real-time systems need bounded response.
P3.12 Fork: Stem: possible output of successful `fork()`. A: two lines print: 0 in the child and child PID in the parent; order is scheduler-dependent.

Chapter 3 Fast Recall

- Process = program in execution; PCB makes pause/resume possible.
- Ready lacks CPU; Blocked lacks an event; Suspended is out of main memory.
- Preemption takes CPU from a process before it yields.
- Interrupts are external; traps are instruction-caused; supervisor calls request OS services.
- A mode switch can stay in the same process; a process switch always changes processes.
- Context switching saves old PCB state and restores the selected process state.

4 Threads

Process vs Thread

Process: unit of resource ownership and protection; owns address space, process image, open files, I/O resources, privileges, and IPC links.
Thread: unit of scheduling/execution; has state, PC, registers, stack, priority, context, and thread-local data. Threads in one process share memory and resources.
Multithreading: multiple concurrent execution paths inside one process. **PCB** stores process resources; **TCB** stores thread execution context.
Advantages: faster create/terminate/switch, cheaper communication, better responsiveness, overlapped I/O and computation, true parallelism on multiprocessors.

States, Operations, Synchronization

Thread states: Running, Ready, Blocked. Suspend states are usually process-level because all threads share the process address space.
Operations: spawn creates a ready thread; block waits for an event; unblock moves to Ready; finish releases context and stack.
Synchronization: required because shared address space/resources can cause data races, lost updates, and corrupted data structures.

ULT, KLT, Combined

ULT: managed by a user-level thread library; kernel sees only the process. Pros: no kernel-mode switch, application scheduling, portable. Cons: blocking syscall may block all process threads; pure ULT cannot exploit multiprocessing.
KLT: managed and scheduled by the kernel. Pros: one blocked thread need not

block the process; threads can run on multiple CPUs; kernel code can be multithreaded. Cons: higher switch overhead.

Jacketing: wrap a blocking system call so the thread library can test/yield and avoid blocking the whole process.

Combined: map many ULTs onto some KLTs/LWPs to mix fast user-level management with kernel parallelism.

Chapter 4 Review Q&A

R4.1 PCB/TCB: Stem: which PCB elements move to TCB. A: TCB gets thread state, PC, registers, stack pointer, priority, scheduling info, private data; PCB keeps PID, address space, memory, files/I/O, privileges, IPC.

R4.2 Switch cost: Stem: why thread mode switch may be cheaper. A: same-process threads keep the same address space and resources; only thread context is saved/restored.

R4.3 Process traits: Stem: two independent process characteristics. A: resource ownership and scheduling/execution.

R4.4 Uses: Stem: four thread uses. A: foreground/background work, asynchronous processing, faster execution by overlap/parallelism, modular program structure.

R4.5 Shared resources: Stem: resources shared by process threads. A: address space, code, global data, heap, files, I/O resources, privileges; each thread has own PC, registers, stack, state.

R4.6 ULT pros: Stem: three ULT advantages over KLTs. A: no kernel switch for thread ops, application-specific scheduling, portability.

R4.7 ULT cons: Stem: two ULT disadvantages. A: blocking syscall may block the whole process; pure ULT cannot use multiple CPUs in parallel.

R4.8 Jacketing: Stem: define jacketing. A: wrapper turns a blocking syscall into nonblocking behavior for user-level thread scheduling.

Chapter 4 Problem Q&A

P4.1 Same-process switch: Stem: same-process vs different-process thread switch. A: yes; same-process switch avoids address-space/resource switch.

P4.2 ULT blocking: Stem: why one blocking ULT blocks all. A: kernel is unaware of ULTs and blocks the whole process on a blocking syscall.

P4.3 OS/2 sessions: Stem: remove sessions, bind UI to processes. A: lose grouping of several processes under one UI; assign memory/files/resources to processes, not threads.

P4.4 1:1 on uniprocessor: Stem: why faster than single thread. A: when one thread waits for I/O, another ready thread can run; CPU use and responsiveness improve.

P4.5 Process exit: Stem: process exits while threads run. A: no thread continues; threads depend on the process address space/resources.

P4.6 OS/390 ASCB/TCB: Stem: split global ASCB and local TCB. A: separates address-space management from task execution and reduces global kernel data.

P4.7 count_positives: Stem: function and two-thread risk. A: counts positive list values into a global counter; unsynchronized shared counter creates data-race risk.

P4.8 Optimized code: Stem: compiler caches global counter. A: local-register copy can overwrite another thread update, causing lost update.

P4.9 Pthreads: Stem: two threads increment `myglobal`; output 21. A: unexpected race; read-add-write is not atomic, so increments are lost.

P4.10 Solaris yield: Stem: yield to same priority only. A: a higher-priority runnable thread would normally be selected by the scheduler; yield donates only among equal-priority peers.

P4.11 Solaris ULT/LWP: Stem: many ULTs per LWP and state diagrams. A: fewer LWPs reduce kernel overhead; ULT state and LWP state differ because user library and kernel schedule different entities.

P4.12 Linux uninterruptible: Stem: rationale for Uninterruptible state. A: protect kernel/device consistency while waiting for critical I/O/resource events; wake only when event completes.

Chapter 4 Fast Recall

- Process owns resources; thread executes code.
- Threads share process memory/resources but keep private PC, registers, stack, and state.
- ULT is fast and portable but weak on blocking and multiprocessing.
- KLT costs more but supports kernel scheduling, blocking isolation, and CPU parallelism.
- Race condition = unsynchronized shared read/write; lost update is the classic symptom.

5 Concurrency and Semaphores

Concurrency Basics

Concurrency: multiple processes or threads make progress in the same time interval. On one CPU execution is interleaved; on many CPUs it may overlap in parallel.

Main hazards: shared global resources, hard resource allocation, and nondeterministic bugs(program hard to reproduce).

Race condition: final result depends on timing of unsynchronized shared read-/write.

Critical section: code that accesses a shared resource. **Mutual exclusion**: at most one process may execute the corresponding critical section for the same resource.

Interaction and OS Concerns

Unaware	competition; issues: mutual exclusion, deadlock, starvation.
Indirectly aware	cooperation by sharing; add data coherence risk.
Directly aware	cooperation by communication; no shared-data mutex, but deadlock/starvation still possible.

OS duties: track processes, allocate/deallocate resources, protect data/resources, and ensure speed independence.

Mutual Exclusion Requirements

- Only one process in a critical section for a resource.
- A halted noncritical process must not block others.
- No deadlock or starvation.
- If the critical section is free, entry is not delayed.
- Assume no process speed or processor count.
- A process stays in the critical section only finite time.

Semaphores

Semaphore: integer synchronization variable accessed only by atomic operations: initialize, `semWait`/P/down, `semSignal`/V/up.

Count meaning: $s > 0$ means available units; $s = 0$ means none free; $s < 0$ means $|s|$ blocked waiters.

Binary semaphore: value 0/1, often mutex-like. **General/counting semaphore**: integer count for multiple resources or ordering.

Mutex vs binary semaphore: a mutex is unlocked by its locker; a binary semaphore may be waited by one process and signaled by another.

Strong semaphore: FIFO wait queue.

Weak semaphore: no specified release order; starvation is possible.

Implementation: `semWait` and `semSignal` must be atomic; use hardware support, compare-and-swap, short interrupt disable on uniprocessors, or software algorithms.

Producer/Consumer Patterns

Infinite buffer: `n=0` counts items; `s=1` protects the buffer. Producer: wait s, append, signal s, signal n. Consumer: wait n, wait s, take, signal s.

Bounded buffer: `e=sizeofbuffer`, `n=0`, `s=1`. Producer: wait e, wait s, append, signal s, signal n. Consumer: wait n, wait s, take, signal s, signal e.

Never wait on the resource count while holding the buffer mutex: producer wait s then e, or consumer wait s then n, can deadlock.

Chapter 5 Review Q&A

R5.1 Design issues: Stem: four concurrency design issues. A: process communication, resource sharing/competition, synchronization, processor-time allocation.

R5.2 Contexts: Stem: three contexts where concurrency arises. A: multiple applications, structured applications, OS structure.

R5.3 Basic requirement: Stem: requirement for concurrent execution. A: enforce mutual exclusion for critical resources, with correct synchronization.

R5.4 Awareness: Stem: three degrees of process awareness. A: unaware/competition; indirectly aware/cooperation by sharing; directly aware/cooperation by communication.

R5.5 Compete vs cooperate: Stem: distinction. A: competing processes only contend for resources; cooperating processes work together by shared data or messages.

R5.6 Control problems: Stem: three competing-process problems. A: mutual exclusion, deadlock, starvation.

R5.7 Mutex requirements: Stem: list requirements. A: one-at-a-time, noncritical halt harmless, no deadlock/starvation, no needless delay, no speed/CPU assumptions, finite CS time.

R5.8 Semaphore ops: Stem: operations. A: initialize, `semWait`/P, `semSignal`/V; no direct arbitrary access.

R5.9 Binary/general: Stem: difference. A: binary is 0/1; general/counting is integer-valued for multiple units or synchronization.

R5.10 Strong/weak: Stem: difference. A: strong uses FIFO waiter release; weak gives no release-order guarantee.

R5.11 Monitor: Stem: monitor. A: high-level synchronization module encapsulating shared data/procedures; only one process active inside, with condition variables for waiting.

R5.12 Messages: Stem: blocking vs nonblocking. A: blocking send/receive waits for completion or message; nonblocking returns immediately after queueing or status check.

R5.13 Readers/writers: Stem: conditions. A: many readers may read together; writers need exclusive access; policies must avoid reader or writer starvation.

Chapter 5 Problem Q&A

P5.1 Multi vs multiprocessing: Stem: two concurrency differences. A: multiprogramming interleaves one CPU and can protect short regions by disabling interrupts; multiprocessing has true overlap, so one-CPU interrupt disable is insufficient.

P5.3 Race traces: Stem: print `x is 10` or `x is 8`. A: interleave the `if` test and `printf` to print 10; split load/decrement/store and load/increment/store to overwrite with stale 8.

P5.4 Tally bounds: Stem: two `total()` processes, $n = 50$. A: final `tally` is 2 to 100; with k processes, upper $50k$, lower can still be 2 for $k \geq 2$.

P5.5 Busy wait: Stem: always less efficient than blocking? A: no; short waits may be cheaper than block/context-switch/restart overhead; long waits should block.

P5.12 Sem definitions: Stem: negative-count vs nonnegative-count semaphore definitions. A: equivalent to programs because waiters are represented either by negative count or by the queue; users cannot inspect internals.

P5.16 General via binary: Stem: flaw and fix. A: binary `delay` can lose wakeups; use pass-the-baton: if `semSignal` wakes a waiter, it signals `delay` instead of releasing `mutex`; awakened waiter releases `mutex`.

P5.19 Consumer deadlock: Stem: move empty-test inside mutex. A: consumer can hold s while waiting on `delay`; producer then waits for s, so both block.

P5.20 Faster infinite buffer: Stem: reduce semaphore overhead. A: let $n = -1$ mean empty and consumer waiting; producer signals `delay` only when append changes n from -1 to 0.

P5.21 Swap order: Stem: bounded-buffer statement swaps. A: swapping wait e/s or wait n/s can deadlock; swapping signal s/n or signal s/e preserves correctness but may add waiting.

P5.22 Trace table: Stem: fill bounded-buffer chart. A: blank labels: Pa p2, Pb p2, Pb p3, Cb c2, Pc p2, Cb c3, Pa p3, Pc p3, Pa

p2, Pa p3; use FIFO queues; final state full = 3, empty = 0, buffer = XXX.

Chapter 5 Fast Recall

- Race condition = timing-dependent result on shared data.
- Protect the code that touches shared state, not just the variable name.
- P/down may block; V/up may wake zero or one waiter.
- Semaphore waits before mutex waits in producer/consumer resource tests.
- Strong semaphore fairness comes from FIFO; weak semaphore may starve.

6 Deadlock and Starvation

Resources and Deadlock Conditions

Reusable resource: not depleted by use; released and used again, e.g., CPU, memory, files, devices, semaphores.

Consumable resource: produced and consumed; after use it is gone, e.g., messages, signals, interrupts, buffer information.

Deadlock: processes are permanently blocked because each waits for an event/resource held by another process. **Starvation**: a ready/requesting process waits indefinitely because others keep being favored.

Mutual exclusion	resource can be held by only one process.
Hold and wait	process holds resources while requesting more.
No preemption	resource cannot be forcibly taken away.
Circular wait	closed chain: each process waits for a resource held by the next.

First three conditions make deadlock possible; adding circular wait creates deadlock.

Handling Strategies

Prevention: make at least one deadlock condition impossible. Examples: request all resources at once, allow preemption where safe, or impose a global resource order.

Avoidance: grant a request only if the resulting state is safe. Needs each process's maximum claim.

Detection: grant resources more freely, periodically test the current state, then recover by aborting, rollback, or preemption.

Banker's Algorithm

$$\text{Need} = \text{Claim} - \text{Allocation}$$

For request R_i: require $R_i \le Need_i$ and $R_i \le Available$; pretend to allocate; grant only if a safe sequence exists.

Safe state: some process completion order can satisfy all remaining needs. **Unsafe state**: no guaranteed safe sequence; deadlock may occur but has not necessarily occurred.

Avoidance vs Detection

Avoidance	before allocation; uses Claim/Max, Allocation, Available; tests $Need_i \le Available$; failure means unsafe.
Detection	after allocation; uses Request Q, Allocation, Available; tests $Q_i \le W$; unmarked at end means deadlocked.

Detection sketch: set $W = Available$; mark processes with zero allocation; repeatedly find unmarked $Q_i \le W$, mark it, set $W = W + A_i$. Remaining unmarked processes are deadlocked.

Chapter 6 Review Q&A

R6.1 Resource examples: Stem: reusable vs consumable resources. A: reusable: CPU, memory, files, devices, semaphores; consumable: messages, signals, interrupts, buffer data.

R6.2 Possible deadlock: Stem: three conditions for deadlock possibility. A: mutual exclusion, hold and wait, no preemption.

R6.3 Actual deadlock: Stem: four deadlock conditions. A: mutual exclusion, hold and wait, no preemption, circular wait.

R6.4 Prevent hold-wait: Stem: hold-and-wait prevention. A: require a process to request all needed resources at once, or release held resources before requesting more.

R6.5 Prevent no-preempt: Stem: two no-preemption attacks. A: if a request fails, release held resources and retry; or preempt resources from another holder when state can be restored.

R6.6 Prevent circular wait: Stem: circular-wait prevention. A: impose a total order on resource types and request only in increasing order.

R6.7 Three strategies: Stem: prevention vs avoidance vs detection. A: prevention breaks a condition; avoidance refuses unsafe grants; detection finds existing deadlocks and recovers.

Chapter 6 Problem Q&A

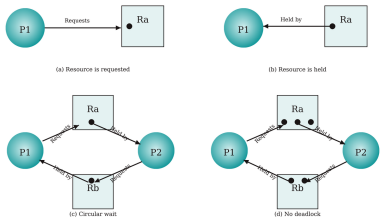
P6.1 Traffic deadlock: Stem: show four conditions in Fig. 6.1a. A: each road quadrant is exclusive; cars hold one quadrant while waiting; cars cannot be preempted; cars wait in a cycle.

P6.2 Apply strategies: Stem: prevention/avoidance/detection for Fig. 6.1. A: traffic lights/order break circular wait; controller allows only safe entry; detection notices blocked cycle and backs/removes one car.

P6.3 Six paths: Stem: narrate Fig. 6.3 paths. A: Q-first, P-then-Q, mixed Q-holds-B/P-releases-A, mixed after A release, P-holds-B/Q-waits, or P-first; all avoid a fatal region.

P6.4 No deadlock: Stem: justify Fig. 6.3. A: P releases A before requesting B; Q may hold B and wait for A, but P does not hold A while waiting for B, so no circular wait.

P6.5 Banker state: Stem: verify, need, safe sequence, P5 request. A: allocated total 9346, so Available 6354. Need rows: P0 7534, P1 2122, P2 3442, P3 2331, P4 4121, P5 3433. Safe order: P1,P2,P0,P3,P4,P5. Granting P5 3233 leaves 3121 and no process can finish; deny.



P6.6 RAG/order: Stem: code with P0 A,B,C; P1 D,E,B; P2 C,F,D. A: deadlock cycle P0 waits C held by P2; P2 waits D held by P1; P1 waits B held by P0. Prevent by global order A<B<C<D<E<F: P0 A,B,C; P1 B,D,E; P2 C,D,F.

P6.11 Memory banker: Stem: total 150, P1 70/45, P2 60/40, P3 60/15; add P4. A: initial Available = 50. P4 hold 25 gives Available 25 and safe order P1,P2,P3,P4; grant. P4 hold 35 gives Available 15 and no need fits; deny.

P6.12 Banker usefulness: Stem: evaluate banker's algorithm in an OS. A: good theory and predictable-resource systems; often impractical because max claims must be known, resources/processes vary, each request costs a safety test, and concurrency may be reduced.

P6.15 Min available: Stem: one resource, $C = (3, 2, 9, 7)$, $A = (1, 1, 3, 2)$. A: Need = $(2, 1, 6, 5)$; minimum Available is 3, e.g., P2,P1,P4,P3.

P6.16 Rankings: Stem: compare six deadlock methods by concurrency and efficiency. A: concurrency high to low: detect+kill, detect+rollback, banker, ordering, restart-on-wait, reserve-all. Rare-deadlock efficiency: ordering, reserve-all, detect+kill, restart-on-wait, banker, detect+rollback; if deadlocks are frequent, detection recovery becomes less attractive.

Chapter 6 Fast Recall

- Prevention breaks conditions; avoidance checks future safety; detection checks current deadlock.
- Unsafe does not mean deadlocked; it means no guaranteed safe completion order.
- Banker's core loop: find $Need_i \le Available$, finish P_i, then add back $Allocation_i$.
- Detection core loop uses current requests Q_i, not maximum claims.
- Global resource ordering prevents circular wait but can reduce flexibility.

7 Memory Management

Requirements and Addresses

Memory management: allocates main memory among active processes and supports efficient multiprogramming.

Requirements: relocation, protection, sharing, logical organization, and physical organization.

Logical address: program-visible address. **Relative address**: logical offset from a known point. **Physical address**: real main-memory address.

Dynamic relocation: hardware maps relative to physical address, commonly $physical = base + relative$; bounds/limit checks enforce protection.

Partitioning

Fixed partitioning	memory split at system generation; simple; causes internal fragmentation and limits active processes.
Dynamic partitioning	allocate exactly requested size; no internal fragmentation; causes external fragmentation and may need compaction.
First-fit	first large enough hole; usually fast and good.
Next-fit	resume search from last placement; often uses later holes.
Best-fit	smallest large enough hole; often leaves many tiny holes.

Internal fragmentation: wasted space inside an allocated block. **External fragmentation**: free space split into noncontiguous holes.

Buddy System and Paging

Buddy system: block sizes are powers of two; split on allocation and coalesce free buddies on release. Buddy address: $buddy_k(x) = x \oplus 2^k$.

Paging: process logical space is split into fixed-size pages; physical memory is split into same-size frames. Pages need not be contiguous.

Page table: maps page number to frame number. Logical address = $(page, offset)$; physical address = $(frame, offset)$.

$p = \lfloor a/s \rfloor, \quad d = a \bmod s, \quad PA = f \times s + d$

Paging has no external fragmentation; only the last page may have internal fragmentation.

Chapter 7 Review Q&A

R7.1 Requirements: Stem: memory-management requirements. A: relocation, protection, sharing, logical organization, physical organization.

R7.2 Relocation: Stem: why relocation is desirable. A: processes can be loaded, swapped, or compacted into different physical locations without changing program ad-

dresses.

R7.3 Compile protection: Stem: why protection cannot be enforced at compile time. A: final physical locations, sharing, and relocation are run-time facts, so run-time checks are needed.

R7.4 Sharing: Stem: why share memory regions. A: shared libraries/code, shared data, IPC, common files/buffers, and memory savings.

R7.5 Unequal fixed partitions: Stem: advantages. A: better match to process sizes, less internal waste, and support for both small and larger processes.

R7.6 Fragmentation: Stem: internal vs external. A: internal is unused space inside an allocation; external is separated free holes outside allocations.

R7.7 Address types: Stem: logical/relative/physical. A: logical is program-visible; relative is an offset; physical is the real main-memory location.

R7.8 Page/frame: Stem: difference. A: a page is a fixed-size logical-process block; a frame is the same-size physical-memory block.

R7.9 Page/segment: Stem: difference. A: pages are fixed-size and system-managed; segments are variable-size logical units such as code, data, or stack.

Chapter 7 Problem Q&A

P7.1 Objectives: Stem: Section 2.3 objectives vs Section 7.1 requirements. A: isolation/protection, allocation/relocation, modularity/logical organization, access control/sharing, and storage hierarchy/physical organization cover the same concerns.

P7.2 Partition pointer: Stem: 2^{16}-byte partitions in 2^{24}-byte memory. A: $2^{24}/2^{16} = 2^8$ partitions, so pointer needs 8 bits.

P7.3 50 percent rule: Stem: holes vs segments in dynamic partitioning. A: in equilibrium, random deletion/allocation gives about one hole per two allocated segments, so holes $\approx n/2$.

P7.4 Search length: Stem: average free-list search. A: best-fit checks all holes unless size-sorted; first-fit and next-fit scan roughly half the list on average.

P7.5 Worst-fit: Stem: largest hole placement. A: may avoid tiny leftovers but destroys large holes; search is full list unless kept largest-first.

P7.6 Diagram: Stem: X is last 2 MB placement; then one swap-out. A: under low-address splitting, swapped-out max = 1 MB; X split a 7 MB hole; next 3 MB: best-fit exact 3 MB hole, first-fit first 4 MB hole, next-fit 5 MB hole after X, worst-fit 8 MB hole.

P7.7 Buddy sequence: Stem: 1 MiB buddy requests 70,35,80; returns A; request 60; returns B,D,C. A: A=128 KiB, B=64 KiB, C=128 KiB, D=64 KiB; after Return B: free A128, free B64, used D64, used C128, free 128, free 512; final all coalesces to 1 MiB free.

P7.8 Buddy address: Stem: block address `011011110000`. A: size 4 buddy `011011110100`; size 16 buddy `011011100000`.

P7.9 Buddy formula: Stem: general buddy expression. A: $buddy_k(x) = x \oplus 2^k$; equivalently flip bit k.

P7.10 Fibonacci buddy: Stem: use Fibonacci sizes. A: yes; split F_{n+2} into F_{n+1} and F_n. Advantage: less internal fragmentation; cost: harder buddy management.

P7.11 PC address: Stem: relative vs absolute PC. A: relative PC is preferable because saved execution state remains relocation-independent; hardware translates on fetch.

P7.12 Paging bits: Stem: 2^{32} physical bytes, page 2^{10}, 2^{16} logical pages. A: logical address 26 bits; frame 1024 bytes; frame field 22 bits; table 2^{16} entries; PTE 23 bits with valid bit.

P7.13 Translate: Stem: logical `0001010010111010`. A: paging: page 20, offset 186, frame 5 gives `0000010110111010`. Segmentation: seg 5, offset 186, base $4+4096\times5$, physical 20670 = `0101000010111110`.

P7.14 Seg table: Stem: translate $(seg, offset)$. A: (0,198)→ 858; (2,156)→ 378; (1,530)→fault; (3,444)→ 1440; (0,222)→ 882.

P7.15 Compaction: Stem: compaction fraction. A: $F \geq (1-f)/(1+kf)$, $k = t/(2s) - 1$. For $f = .2, t = 1000, s = 50$, $k = 9$, so $F \geq 0.8/2.8 \approx 0.286$.

Chapter 7 Fast Recall

- Fixed partitions waste inside; dynamic partitions waste between holes.
- Compaction fights external fragmentation but costs CPU and needs relocation.
- Buddy allocation rounds up to a power of two, splits downward, and coalesces upward.
- Paging maps page number to frame number; the offset is copied unchanged.
- Simple paging requires all pages resident; virtual-memory paging can load pages on demand.